Uploaded a file

Here's the latest code. Please review the whole thing, including but not limited to code, correctness of documentation, and accessibility.

I'll review the project broadly: build/config structure, app code, tests if present, docs, and accessibility. I'll focus first on issues that could affect beta/readiness, then separate nice-to-haves.

I've unpacked the project and can inspect the source, but this environment does not have `xcodebuild`, so I can't compile or run the test suite here. I'm doing a static review instead and will flag anything that needs Xcode verification separately.

Thought for 1m 12s >

I reviewed the zip statically. I could not build/run tests here because `xcodebuild` is unavailable.

Overall: this is in good shape for an internal beta, but I'd fix these before wider sharing:

1. **README is effectively empty/broken**
   `README.md` only contains `# NextSeason` with no newline. Portfolio reviewers need this filled in.
2. **Analytics is not actually beta-useful yet**
   `AnalyticsService` logs to `os.Logger`, which is good architecture, but it will not give you aggregated TestFlight beta behavior unless you collect device logs. The docs imply analytics will inform beta priorities; either add a real provider later or reword docs as "local instrumentation scaffold."
3. **Visible "Tap here for Actor Name Analytics" text should go before any human beta**
   It is documented as portfolio-only cleanup, but it is user-visible and confusing. I would remove it now, not later.
4. **Theme switcher / icon switching should be DEBUG-only before beta**
   Leaving it visible is fine for palette testing, but automatic alternate-icon changes can feel weird and prototype-ish. Wrap the overlay and icon sync in `#if DEBUG` unless testers are specifically evaluating themes.
5. **Release Readiness has stale/conflicting first-run notes**
   It says the search idle state is still a remaining opportunity, but the "Try an Example" implementation is already done.
6. **Project/package hygiene**
   The zip includes `.git`, `.DS_Store`, `xcuserdata`, and `UserInterfaceState.xcuserstate`. Not necessarily committed, but clean those before sharing a source archive. Also `.gitignore` has `.DS_STORE`; it should be `.DS_Store`.
7. **Accessibility looks thoughtfully implemented, but still needs manual audit**
   Good: decorative posters hidden, row labels combined, track buttons labeled. Still verify Dynamic Type, VoiceOver row/button navigation, Increased Contrast, and the undo toast.

No obvious code-level correctness blocker jumped out from the static review. My top recommendation: remove the beta-only analytics tap target and DEBUG-wrap the theme switcher before sending this to friends.

> The theme switcher is actually not DEBUG-wrapped on purpose. I want to get people's feedback on which theme they prefer.

That makes sense. I'd revise that finding to:

Keep the theme switcher visible for this feedback round, but label it clearly as a beta feedback feature, and remove or hide it before portfolio/App Store release.

I'd still consider disabling the **automatic app icon change** part unless you specifically want feedback on icons too. The visual theme picker is useful; surprise icon changes may distract testers.

Good catch. I think you're right. The purpose of **First-Run Experience** should be narrowly focused on whether a brand new user can get started successfully. Things like removing beta controls, cleaning up analytics hooks, and polishing the portfolio all belong elsewhere now.

I'd trim it down to something like this:

Markdown

```
### First-Run Experience

The first-run experience is ready for beta testing.

**Implemented**
- Empty search state provides guidance and "Try an Example" buttons to help new users get started.
- Empty Watchlist explains its purpose and how to add shows.
- Search is immediately discoverable, reducing the chance of a confusing blank screen.

**Beta Validation**
- Confirm that new users understand they should begin by searching for a show.
- Observe whether testers naturally discover how to add a show to their Watchlist.
- Ask whether the example searches are helpful or if different examples would better demonstrate th
```

I actually like this better than my previous suggestion because it's answering one very specific question: **"Can a first-time user figure out how to use the app?"** Everything else—portfolio polish, removing the analytics label, hiding the theme picker, App Store assets, README improvements—is already covered (or should be covered) under **Portfolio Readiness** or other sections of the document. That keeps each section focused and avoids duplication.

> Please recommend what changes you feel should be made to analytics. The goal was to avoid using a third party service, particularly one that's not free.

I actually think your instincts are good here. For an MVP and portfolio project, I would **not** add a third-party analytics SDK. It increases privacy disclosures, maintenance, app size, and dependency management, while providing relatively little value for a small TestFlight audience.

Instead, I'd evolve your existing analytics into something that gives you useful information without sending data anywhere.

My recommendation: Three layers

Layer 1: Structured event logging (keep this)

Instead of arbitrary log strings, define a strongly-typed event enum.

Swift

```
enum AnalyticsEvent {
    case appLaunched
    case searchStarted(queryLength: Int)
    case searchCompleted(resultCount: Int)
    case searchNoResults
    case showOpened(id: Int)
    case addedToWatchlist(id: Int)
    case removedFromWatchlist(id: Int)
    case notificationEnabled
    case notificationDisabled
}
```

Your service simply logs these consistently.

That gives you:

- cleaner code
- compile-time safety
- one place to add/remove events

Layer 2: Local counters

This is the piece I think is missing.

Persist simple counters in `UserDefaults`.

For example:

```
Searches performed: 52

Successful searches: 48

No-result searches: 4

Shows added: 17

Shows removed: 6

Notifications enabled: yes

Example searches tapped: 3
```

No personal information.

No cloud.

No SDK.

Just useful diagnostics.

Layer 3: Export diagnostics

This is the feature I think would help you the most during TestFlight.

Add a hidden screen (or long-press version number) showing:

```
Analytics Summary

Searches: 52

No Results: 4

Shows Viewed: 73

Watchlist Adds: 17

Notification Toggles: 2

App Launches: 19
```

with

```
Copy
```

or

```
Share…
```

That exports something like

```
NextSeason Analytics

Version 1.0 (24)

App launches: 19

Searches: 52

Successful searches: 48

No-result searches: 4

Example searches: 3

Watchlist adds: 17

Watchlist removals: 6

Notifications enabled: true
```

Now when a beta tester says

> "Search doesn't work."

you can ask them to

> Settings → Diagnostics → Share Analytics

and immediately know

- did they actually search?

- were they getting zero results?
- did they ever add anything?
- did notifications get enabled?

without collecting anything automatically.

Events I'd collect

I wouldn't collect every tap.

I'd focus on decisions.

Search

- Search submitted
- Result count
- No results
- Example search used

Watchlist

- Add
- Remove
- View details
- Watchlist opened

Notifications

- Permission requested
- Permission granted
- Reminder scheduled
- Reminder cancelled

Settings

- Theme changed
- Alternate icon changed

Beta

Since you specifically want theme feedback:

```
Theme selected

Light

Dark

System

Purple

etc.
```

At the end of beta you'll actually know which themes people chose.

Things I would NOT log

These aren't likely to be actionable for an app of this size:

- every row tap
- scrolling
- actor taps
- poster taps
- every navigation transition
- typing every character

Those generate a lot of noise without helping you make product decisions.

The one thing I *would* add

I remember our earlier discussion about the actor names not doing anything yet.

Originally I suggested logging those taps.

I don't think I'd do that anymore.

Instead, if you really want to know whether people expect actors to be tappable, I'd make the actor names **buttons** during beta that simply display a small message like:

> Actor details are planned for a future release.

and increment a local counter.

That measures genuine user intent much better than an invisible gesture recognizer, because users receive feedback instead of wondering whether they missed the tap target.

For a portfolio project, I actually think this approach is stronger than integrating a service like Firebase. It demonstrates that you've designed analytics deliberately around the questions you need answered, rather than instrumenting everything because a third-party SDK makes it easy. It also keeps the app privacy-friendly, offline-capable, and free of external dependencies while still giving you actionable data during your TestFlight beta.

Please put all your analytics suggestions into a Markdown file I can add to the project (and give to Cursor)

I put everything into a standalone Markdown document suitable for adding directly to the project and handing to Cursor.

You can download it here:

Analytics Recommendations.md

I also organized it in roughly the same style as the other project planning documents, with:

- Goals and guiding philosophy
- Structured analytics architecture
- Local counters
- Hidden diagnostics/export screen
- Recommended events
- Events intentionally *not* tracked
- Privacy principles
- Future expansion path

One thing I'd add, having thought about this more: I think this analytics strategy actually becomes a **portfolio strength**. Many portfolio projects either have no analytics at all or immediately pull in Firebase. Yours would demonstrate that you intentionally designed an analytics system around privacy, offline operation, and actionable metrics—exactly the kind of engineering tradeoff experienced developers make.

‹/›